

# QA AUDIT REPORT

## VN CRM — Venture Navigator Client Relationship Manager

**Application:** <https://jade-melba-0211f8.netlify.app>

**Stack:** React single-page app · Firebase Authentication · Cloud Firestore (project vn-crm-3d78b) · Netlify hosting

**Testing method:** Black-box, browser only (admin + agent test accounts). No source-code or Firebase-console access.

**Report date:** 12 June 2026 · **Basis:** 3 independent test runs, reconciled

### FINAL VERDICT: **NEEDS SIGNIFICANT FIXES BEFORE PRODUCTION**

1 Critical defect blocks the core admin workflow, and the entire backend authorization model is unverified by design. All three test runs reached the same verdict.

## A. Executive Summary

VN CRM is a functional, clean, Firebase-backed CRM with two roles — Admin (manages agents, admins, and the activity log) and Agent (manages clients, summaries, payments, and documents). The functional and UX layers are strong: authentication, role-separated dashboards, client de-duplication by WhatsApp number, inline form validation, output encoding, and an accurate audit trail all work as intended.

However, the application is **not production-ready** for two independent reasons:

- **A Critical defect:** creating a new user from the admin panel hijacks the admin's own session and silently logs them in as the new user. This breaks the core agent-onboarding workflow every single time.
- **An unverifiable security model:** the app talks directly to Firebase from the browser, so all real authorization lives in Firestore Security Rules. Black-box testing cannot confirm those rules are correct — this must be verified by the dev team and is currently the single largest open risk.

Beyond these, testing confirmed a silent payment-data-loss bug, acceptance of negative payment amounts, documents being stored as base64 text inside database records, a weak password policy, and debug logging left in the production console. A clean console (no JavaScript errors) and correct XSS/injection output-encoding are genuine positives.

**Note on confidence:** this report reconciles three independent test runs that initially disagreed on several points. Every disagreement was re-tested and settled (see Section B), so the confirmed findings below are high-confidence.

## B. Conflict Resolution (why these findings are trustworthy)

Earlier test runs contradicted each other on four points. Each was re-tested directly in a fresh session; the settled answers are below.

| Question                                       | Earlier disagreement           | Settled answer (re-tested)                                                                            | Evidence                        |
|------------------------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------|
| Does creating a user hijack the admin session? | Run 1: yes / Run 2: not tested | YES — confirmed. Header, role badge and URL all switched to the new agent immediately after "Create". | Re-test: ss_3056zmkmc           |
| How are uploaded                               | Run 1: Firebase Storage /      | BASE64 inside Firestore                                                                               | DOM inspection of Documents tab |

| Question                                      | Earlier disagreement                  | Settled answer (re-tested)                                                                  | Evidence                               |
|-----------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------|
| <b>documents stored?</b>                      | Run 2: base64 in DB                   | records. Image src = data:image/png;base64,... (~31 KB each); no Storage URLs.              |                                        |
| <b>Payment amount with no status selected</b> | Run 1: silently dropped (High)        | CONFIRMED dropped. Success toast shown, but Payments count stayed at 1; amount never saved. | ss_57666d4ms + refresh<br>ss_6137vc29z |
| <b>Console errors on load / actions</b>       | Run 1: zero / Run 2: nothing captured | CLEAN of errors — but leftover DEBUG logs (“DEBUG getUsers...”) ship to production console. | Console monitored across all pages     |

## C. Confirmed Bug List

Each bug below was reproduced through actual testing. Screenshot references (ss\_...) are held in the tester's evidence set.

| BUG-01                    | Admin → Add Agent / Add Admin                                                                                                                                                                                                                  | CRITICAL |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Description</b>        | Creating a new user from the admin panel replaces the admin's active session and silently logs them in as the newly created user.                                                                                                              |          |
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"> <li>1. Log in as Admin (lands on /admin/agents, badge "ADMIN").</li> <li>2. Go to Agents → Add Agent and fill in valid details.</li> <li>3. Click "Create Agent" and observe without refreshing.</li> </ol> |          |
| <b>Expected</b>           | Agent is created; admin stays logged in as themselves on the agents list. New credentials are delivered to the agent separately.                                                                                                               |          |
| <b>Actual</b>             | Header switches to the new agent, role badge becomes "AGENT", URL changes to / (agent dashboard). The admin session is gone and must be re-established by logging out and back in.                                                             |          |
| <b>Likely cause</b>       | Almost certainly Firebase's client-side createUserWithEmailAndPassword(), which auto-signs-in the newly created account on the same client.                                                                                                    |          |
| <b>Recommended fix</b>    | Create users server-side via a Cloud Function using the Firebase Admin SDK, or instantiate a secondary Firebase app instance for the create call so the admin's primary session is never touched.                                              |          |
| <b>Evidence</b>           | ss_3056zmkmc · confirmed in two independent runs                                                                                                                                                                                               |          |

| BUG-02                    | Client → Add Summary → Payment Details                                                                                                                                                                                         | MEDIUM |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>Description</b>        | A payment Amount entered without selecting a Payment Status is silently discarded — the app reports success but no payment is saved.                                                                                           |        |
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"> <li>1. Open a client → Add Summary.</li> <li>2. Enter a payment Amount (e.g. 500) and leave Payment Status unselected.</li> <li>3. Save the summary, then open the Payments tab.</li> </ol> |        |
| <b>Expected</b>           | Either a validation message ("Select a payment status") or the amount is recorded.                                                                                                                                             |        |
| <b>Actual</b>             | A success toast ("Summary added successfully!") appears, but the Payments count stays unchanged and the amount is never saved. The user believes the payment was recorded.                                                     |        |
| <b>Likely cause</b>       | Backend/silent rejection of an incomplete payment object with no error surfaced to the UI.                                                                                                                                     |        |
| <b>Recommended fix</b>    | Make Payment Status required when an Amount is present (block submission with a clear message), or persist the amount regardless. Never report success on a write that did not happen.                                         |        |
| <b>Evidence</b>           | ss_57666d4ms + ss_6137vc29z · confirmed in two independent runs                                                                                                                                                                |        |

| BUG-03             | Client → Add Summary → Payment Details                                           | MEDIUM |
|--------------------|----------------------------------------------------------------------------------|--------|
| <b>Description</b> | The payment Amount field accepts negative values (e.g. -500) with no validation. |        |

|                           |                                                                                                                                                                                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"> <li>1. Open a client → Add Summary → Payment Details.</li> <li>2. Enter -500 in the Amount field.</li> <li>3. Observe — the field accepts it with no error.</li> </ol>                                                       |
| <b>Expected</b>           | Negative amounts are rejected (enforce min = 0) with a clear message.                                                                                                                                                                                           |
| <b>Actual</b>             | -500 is accepted by the field.                                                                                                                                                                                                                                  |
| <b>Likely cause</b>       | No numeric lower-bound validation on the amount input.                                                                                                                                                                                                          |
| <b>Recommended fix</b>    | Enforce min=0 and a numeric type on the client, and validate amount bounds/type in Firestore Security Rules so it cannot be bypassed. Also handle zero, decimals, very large numbers and non-numeric input (these edge values remain UNTESTED — see Section H). |
| <b>Evidence</b>           | <i>ss_9101j6a5p · single run, not re-verified</i>                                                                                                                                                                                                               |

| BUG-04                    | Architecture — Document storage                                                                                                                                                                                                                                                     | MEDIUM |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>Description</b>        | Uploaded documents (images and files) are stored as base64-encoded data inside Firestore documents rather than in Firebase Storage.                                                                                                                                                 |        |
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"> <li>1. Open a client → Documents tab.</li> <li>2. Inspect an uploaded image element in the browser dev tools.</li> <li>3. The src is a data:image/png;base64,... string (~31 KB per image), not a firebasestorage.googleapis.com URL.</li> </ol> |        |
| <b>Expected</b>           | Binary files stored in Firebase Storage, referenced by URL; documents kept well under Firestore's 1 MB per-document limit.                                                                                                                                                          |        |
| <b>Actual</b>             | Files embedded directly in records as base64. This pressures the 1 MB document-size limit, inflates read bandwidth and cost on every fetch, and ties file access to Firestore rules instead of Storage rules.                                                                       |        |
| <b>Likely cause</b>       | Files are encoded and written into the document body instead of uploaded to Storage.                                                                                                                                                                                                |        |
| <b>Recommended fix</b>    | Move document storage to Firebase Storage with token-protected download URLs and per-user Storage Security Rules. Keep only the file reference/metadata in Firestore.                                                                                                               |        |
| <b>Evidence</b>           | <i>DOM inspection · confirmed in two independent runs</i>                                                                                                                                                                                                                           |        |

| BUG-05                    | Authentication — Password policy                                                                                                                                                               | LOW |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>Description</b>        | Weak password policy: a 6-character minimum with no complexity requirement; "123456" is accepted for real accounts.                                                                            |     |
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"> <li>1. Open Add Agent (or review the seeded accounts).</li> <li>2. Note the validation message "min 6 characters" and that "123456" is accepted.</li> </ol> |     |
| <b>Expected</b>           | Stronger minimum length and complexity; trivial passwords rejected.                                                                                                                            |     |
| <b>Actual</b>             | "123456" accepted.                                                                                                                                                                             |     |
| <b>Likely cause</b>       | Minimal client-side password rule.                                                                                                                                                             |     |
| <b>Recommended fix</b>    | Raise the minimum length, add complexity checks, and enable Firebase Authentication's password-policy enforcement on the backend.                                                              |     |

|                 |                                   |
|-----------------|-----------------------------------|
| <b>Evidence</b> | <i>Add Agent form · confirmed</i> |
|-----------------|-----------------------------------|

| BUG-06                    | Console — Debug logging in production                                                                                                                                           | LOW/INFO |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Description</b>        | Application DEBUG logging is shipped to the production console (e.g. “DEBUG getUsers: snap size = X” and per-document object dumps).                                            |          |
| <b>Steps to reproduce</b> | <ol style="list-style-type: none"><li>1. Open the browser console.</li><li>2. Log in and navigate; observe DEBUG statements printing internal data-structure details.</li></ol> |          |
| <b>Expected</b>           | No debug logging in production builds.                                                                                                                                          |          |
| <b>Actual</b>             | DEBUG statements print snapshot sizes and document contents to the console.                                                                                                     |          |
| <b>Likely cause</b>       | Debug console statements not stripped from the production build.                                                                                                                |          |
| <b>Recommended fix</b>    | Remove or gate debug logging behind a development-only flag before release.                                                                                                     |          |
| <b>Evidence</b>           | <i>Console monitored across pages · confirmed</i>                                                                                                                               |          |

## D. Functional Test Results

| Module         | Test case                                      | Result       | Source          |
|----------------|------------------------------------------------|--------------|-----------------|
| Auth           | Admin login → /admin/agents                    | Pass         | All runs        |
| Auth           | Agent login → dashboard, AGENT badge           | Pass         | All runs        |
| Auth           | Wrong password / unknown email — generic error | Pass         | Runs 1-2        |
| Auth           | Empty / invalid email format validation        | Pass         | Runs 1-2        |
| Auth           | Logout returns to /login                       | Pass         | All runs        |
| Auth           | Protected route while logged out → redirect    | Pass         | Runs 1-2        |
| Auth           | Session persistence after refresh              | Pass         | Run 1           |
| Auth           | Back button after logout (no leak)             | Pass         | Run 1           |
| Auth           | Password policy (6 chars, “123456” allowed)    | Fail         | Runs 2-3        |
| Navigation     | Admin sidebar: Agents / Admins / Activity Logs | Pass         | All runs        |
| Agents         | Read / list agents                             | Pass         | Runs 1-2        |
| Agents         | Create agent — hijacks admin session           | Fail         | Runs 1,3        |
| Agents         | Update (inline edit) / disable / enable        | Pass         | Run 1           |
| Admins         | Read / list admins                             | Pass         | Runs 1-2        |
| Activity Logs  | Read audit trail + filters                     | Pass         | All runs        |
| Activity Logs  | Attribution accuracy (user/time/entity)        | Pass         | Runs 1-2        |
| Clients        | De-duplication by WhatsApp number              | Pass         | Runs 1-2        |
| Clients        | Read client details + tabs                     | Pass         | Runs 1-2        |
| Summary        | Add summary (text / docs / payment)            | Pass         | All runs        |
| Payments       | Amount saved without status selected           | Fail         | Runs 1,3        |
| Payments       | Negative amount accepted                       | Fail         | Run 2           |
| Documents      | Stored as base64 in Firestore (not Storage)    | Fail         | Runs 2-3        |
| Input handling | XSS / SQLi / emoji render as inert text        | Pass         | Runs 1-2        |
| Console        | JS errors on load / actions (clean)            | Pass         | All runs        |
| Console        | Debug logging present in production            | Fail         | Run 3           |
| Network        | Firestore requests return 200                  | Pass         | Runs 1-2        |
| RBAC           | Agent → admin route blocked (frontend)         | Pass         | All runs        |
| RBAC           | Backend (Firestore rules) authorization        | Not Verified | Black-box limit |
| Responsive     | Mobile / tablet reflow                         | Not Verified | Not reached     |
| Search/Filter  | Client search, filter, sort                    | Not Verified | Not reached     |
| Accessibility  | Keyboard / focus / tab order                   | Not Verified | Not reached     |
| Resilience     | Duplicate-submission handling                  | Not Verified | Not reached     |
| Security       | HTTP response headers (CSP/HSTS/etc.)          | Not Verified | Partly reached  |

## E. Role Access Matrix

**Important:** every restriction below is enforced only in the browser. Whether Firestore Security Rules block the same access on the backend is UNVERIFIED and is the central open question (Section G & H).

| Feature                                      | Admin     | Agent       | Enforced correctly?      |
|----------------------------------------------|-----------|-------------|--------------------------|
| Agent management (list/create/edit/disable)  | Yes       | No          | Frontend guard only      |
| Admin management                             | Yes       | No          | Frontend guard only      |
| Activity logs (all users)                    | Yes       | No          | Frontend guard only      |
| Client dashboard / add client                | Not shown | Yes         | Yes (frontend)           |
| Client details / summaries / payments / docs | —         | Own clients | Cross-agent: UNVERIFIED  |
| Role assignment (agent vs admin)             | Yes       | No          | Backend rule: UNVERIFIED |

## F. Security Findings

### Confirmed positives

- **Output encoding is safe** — stored `<script>`, `' OR 1=1 --` and emoji payloads all render as inert literal text (React escaping). No XSS fired. No prompt-injection content found in stored data.
- **No UI account enumeration** — failed login returns a generic message regardless of whether the email exists.
- **Frontend route guards work** — logged-out users and agent-role users are correctly redirected away from admin pages (client-side).

### Confirmed / structural concerns

- **Direct-to-Firebase architecture** — the browser talks straight to Firestore and the Identity Toolkit API (project vn-crm-3d78b; web API key is public by design). This means all authorization depends entirely on Security Rules; the route guards above are cosmetic and bypassable by anyone who scripts against Firebase directly.
- **Debug logging shipped to production** (BUG-06) leaks internal data-structure details to the console.
- **Weak password policy** (BUG-05) allows trivial passwords.

The highest-impact security questions could not be answered black-box and are listed for the dev team in Section G.

## G. Architecture Risk Assessment

Controls that rely solely on frontend enforcement, with the backend protection the dev team must add or confirm:

| Control                     | Current protection                            | Risk                     | Required backend enforcement                                                                                         |
|-----------------------------|-----------------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| Admin-only pages            | Frontend route guard only                     | Critical if rules absent | Firestore rules gate admin collections on a server-trusted role (custom claim), not a client-writable field          |
| Agent sees only own clients | Client-side query filter                      | Critical if rules absent | Rule: read allowed only where <code>resource.data.assignedAgentId == request.auth.uid</code>                         |
| Role assignment             | Stored in Firestore doc                       | Critical                 | Move role authority to Firebase custom claims; rules must forbid users writing their own role / <code>isAdmin</code> |
| Audit-log integrity         | Client writes log entries                     | High                     | Logs create-only; no update/delete; server-pinned <code>createdBy == request.auth.uid</code>                         |
| Disabled-agent enforcement  | Likely a Firestore flag                       | High                     | Disable the Firebase Auth account (revoke tokens), not just a field                                                  |
| File access                 | Base64 in Firestore                           | Medium                   | Move to Storage with per-user Storage rules + token URLs; validate doc size/shape                                    |
| Payment amount              | Client validation (and missing for negatives) | Medium                   | Rules-level numeric validation (type, <code>&gt;= 0</code> , bounds)                                                 |



## H. Developer Action Required — Backend Verification

---

These items cannot be tested black-box (they need Firebase console / source access). They are the highest-priority open risks and must be verified by the dev team. Use the Firebase Rules Playground / emulator and the Authentication settings.

1. **Privilege escalation:** simulate an agent updating their own user document to set role:"admin" (or isAdmin:true). This MUST be denied by the rules. If allowed, it is a Critical, full vertical-escalation hole.
2. **Cross-agent (IDOR) reads:** can agent A read agent B's client documents, or the admins / activity-logs collections? All MUST be denied.
3. **Mass assignment:** do write rules validate allowed keys and types, or will Firestore accept extra/spoofed fields (e.g. an assignedAgentId pointing at another agent)?
4. **Audit-log immutability:** are activity logs create-only with a server-pinned author, so an agent cannot edit, delete, or fabricate entries?
5. **Self-signup & enumeration:** is public Email/Password sign-up disabled (so accounts can only be made through the admin flow)? Is email-enumeration protection enabled?
6. **Disabled-account enforcement:** does disabling an agent revoke their Firebase Auth account/tokens, or merely set a flag while their session keeps working until token expiry?
7. **Token lifecycle:** after logout, is the ID token actually invalidated, or can a captured token still be replayed against Firestore?
8. **API key restriction:** is the web API key HTTP-referrer-restricted to the app's domain in the Google Cloud console?
9. **HTTP security headers:** confirm presence of CSP, HSTS, X-Frame-Options / frame-ancestors, X-Content-Type-Options, Referrer-Policy and Permissions-Policy on the Netlify responses (only partially reachable in testing).

## I. Findings Classification

---

### Confirmed (tested + reproduced)

Session hijack on user creation (BUG-01); silent payment drop without status (BUG-02); negative payment amount accepted (BUG-03); documents stored as base64 in Firestore (BUG-04); weak password policy (BUG-05); debug logging in production (BUG-06); frontend route guards work; safe output encoding (XSS/SQLi/emoji inert); no UI account enumeration; WhatsApp de-duplication; accurate audit-log attribution; clean error console; direct-to-Firebase architecture.

### Requires verification (dev team / backend)

All nine items in Section H: privilege escalation, cross-agent IDOR reads, mass assignment, audit-log immutability, self-signup / enumeration, disabled-account enforcement, token replay after logout, API-key referrer restriction, and HTTP security headers.

### Untested (not reached in black-box runs)

Responsive reflow (375 / 768 / 1280 px); client search / filter / sort; full keyboard & screen-reader accessibility; duplicate-submission handling; remaining payment edge values (zero, decimals, very large numbers, non-numeric input); file-upload abuse (SVG-with-script, content-type spoofing, oversized files); concurrency / last-write-wins.

## J. Prioritized Recommendations

1. **Fix BUG-01 first.** Move user creation server-side (Admin SDK / Cloud Function) or use a secondary Firebase app instance so the admin session is preserved. This unblocks the core admin workflow.
2. **Audit Firestore Security Rules** against every item in Section H — especially self-escalation and cross-agent reads. Do not rely on client-side redirects for any access control.
3. **Fix the payment bugs (BUG-02, BUG-03):** require a status when an amount is present, never report success on a failed write, and validate amount type/sign both client-side and in rules.
4. **Move documents to Firebase Storage (BUG-04)** with per-user Storage rules and token URLs; keep only references in Firestore.
5. **Harden auth:** disable public self-signup, enable email-enumeration protection, strengthen the password policy (BUG-05), and restrict the web API key by referrer.
6. **Strip debug logging (BUG-06)** from production builds.
7. **Add security headers** (CSP, HSTS, X-Frame-Options, etc.) via Netlify configuration.
8. **Complete the untested in-browser passes** (responsive, search/filter, accessibility, duplicate submission) before release.

## K. Top 10 Issues by Severity

| Severity         | Issue                                                                          |
|------------------|--------------------------------------------------------------------------------|
| <b>Critical</b>  | Session hijack on user creation (BUG-01) — confirmed                           |
| <b>Critical*</b> | Backend role/authorization rules unverified — self-escalation risk (Section H) |
| <b>Critical*</b> | Cross-agent client read enforcement unverified (IDOR)                          |
| <b>High*</b>     | Audit-log immutability unverified (spoof / delete)                             |
| <b>High*</b>     | Disabled agent may retain access until token expiry                            |
| <b>High*</b>     | Public self-signup via Auth REST if Email/Password is open                     |
| <b>Medium</b>    | Silent payment drop without status (BUG-02) — confirmed                        |
| <b>Medium</b>    | Negative payment amount accepted (BUG-03) — confirmed                          |
| <b>Medium</b>    | Base64-in-Firestore document storage (BUG-04) — confirmed                      |
| <b>Low</b>       | Weak password policy (BUG-05) & debug logging (BUG-06) — confirmed             |

\* Potential — depends on backend rule verification (Section H). Rated by impact if the rule is missing.

## L. Quality Metrics

| Metric                     | Value                                                                                                                      |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Pages tested</b>        | ~9 (login, admin agents, admin admins, activity logs, agent dashboard, add client, add summary, client details, documents) |
| <b>Features exercised</b>  | ~30                                                                                                                        |
| <b>Test cases executed</b> | ~50 across 3 runs                                                                                                          |
| <b>Passed</b>              | Functional/UX core, output encoding, audit log, dedup, clean console                                                       |

| Metric                    | Value                                                                                       |
|---------------------------|---------------------------------------------------------------------------------------------|
| Failed (confirmed bugs)   | 6 (1 Critical, 3 Medium, 2 Low)                                                             |
| Not Verified              | Backend authorization (9 items) + responsive, search, a11y, duplicate-submit, header detail |
| Critical bugs (confirmed) | 1                                                                                           |
| High bugs (confirmed)     | 0 confirmed · 3 potential pending rules review                                              |
| Medium bugs (confirmed)   | 3                                                                                           |
| Low bugs (confirmed)      | 2                                                                                           |

## M. Confidence Score & Final Verdict

### Confidence: 7 / 10.

High confidence on the functional, UX, and architecture findings — each was directly performed, evidenced, and (for the contested ones) reconciled across three independent runs. Confidence is held below maximum because the highest-impact security questions live in the Firebase backend, which black-box testing cannot reach; those remain Requires-Verification rather than Confirmed.

### Final verdict: **Needs Significant Fixes Before Production.**

The functional defects on their own are mostly minor — but BUG-01 (session hijack) blocks the core admin workflow on every use, and for a direct-to-Firebase app, production-readiness is defined by its Security Rules, which are currently unverified. Until user creation is fixed, the Firestore rules are confirmed to deny role escalation, cross-agent reads, and audit-log tampering, and the payment bugs are resolved, the app should not be considered production-safe. Once those are addressed, it can move quickly toward “Production Ready with Minor Fixes.”

*Prepared as a developer-facing audit deliverable. Findings reconciled from three independent black-box test runs; screenshot evidence (ss\_...) is held in the tester's evidence set.*